GUIDE 04 · START

Connect to MongoDB

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=04-connect-to-mongodb. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a `find()`-based saved query - with bound parameters, projection and sort - callable as a versioned, validated REST endpoint, exactly like a SQL one.

> **Stable** since 0.37, under the compatibility promise - what that covers, what may still be added and its limits: MongoDB connections.

MongoDB support in QueryAPIGate is **`find()`-only, read-only, full stop** - no aggregation pipelines, no writes, no response caching (yet). Everything below is what's actually built; nothing here is a partial feature with hidden gaps.

Step 1: Add the connection

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "shop-mongo",
  "db": "mongo", "host": "localhost", "port": 27017, "database": "shop",
  "user": "${MONGO_USER}", "password": "${MONGO_PASSWORD}", "active": true
}'
```

Needs `pip install "queryapigate[mongo]"`. `user` / `password` are optional - omit both for an unauthenticated local instance, as this guide's own examples do. `database` selects which database `find()` runs against per call; unlike the SQL dialects, it isn't part of the driver's own connection string.

Test and browse it the same way as any other connection:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections/test -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"db": "mongo", "host": "localhost", "port": 27017, "database": "shop"}'
# {"elapsed_ms": 12.6}

curl http://127.0.0.1:5000/connections/shop-mongo/schema -H 'X-API-Key: demo-key'
# {"tables": [{"name": "orders", "type": "collection", "columns": []}], "truncated": false}
```

Schema introspection lists collection names only - Mongo documents are schemaless, so there are no columns to list the way a SQL table has.

Step 2: Run an ad-hoc `find()`

`POST /execute_mongo` is the Mongo sibling of `/execute_sql` - same idea (run something ad hoc against a connection before deciding to save it), different shape:

shell

```
curl -X POST http://127.0.0.1:5000/execute_mongo -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "collection": "orders",
  "connection_name": "shop-mongo",
  "filter": {"status": ":status"},
  "params": {"status": "paid"},
  "projection": {"customer": 1, "total": 1, "_id": 0},
  "sort": {"total": -1}
}'
```

json

```
[{"customer": "Carol", "total": 87.25}, {"customer": "Alice", "total": 42.5}]
```

`:status` here is the exact same idea as a SQL query's bound `:name` parameter - a placeholder that only ever becomes a real value through `params`, never text spliced into the filter. It works as a whole string leaf anywhere in the filter document, including nested inside an operator:

json

```
{"filter": {"total": {"$gte": ":min_total"}}, "params": {"min_total": 50}}
```

Two operators are rejected outright, on any request: `$where` and `$function` / `$accumulator` run arbitrary server-side JavaScript - the Mongo equivalent of SQL injection into a `WHERE` clause, and the one thing a find-only path must never allow through, so there's no configuration to get right here, they're simply never permitted - in the filter, the projection or the sort. And a parameter's value is a value - a string, number, boolean, null, or a list of those - never a document: `{"name": ":u"}` given `{"$ne": null}` is refused (`param_invalid`), not run as an operator that matches everything.

shell

```
curl -X POST http://127.0.0.1:5000/execute_mongo -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"collection": "orders", "connection_name": "shop-mongo", "filter": {"$where": "true"}}'
# {"error": "'$where' is not allowed - it runs arbitrary server-side JavaScript"}
```

Step 3: Save it as a REST endpoint

Same idea as a SQL saved query - `POST /api/v1/queries`, just with `query_type: "mongo"` and `mongo_*` -prefixed fields instead of `sql`:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "orders_by_status",
  "query_type": "mongo",
  "mongo_collection": "orders",
  "mongo_filter": {"status": ":status"},
  "mongo_projection": {"customer": 1, "total": 1, "_id": 0},
  "mongo_sort": {"total": -1},
  "parameters": {"status": {"type": "str", "enum": ["paid", "pending"], "description": "Order status"}},
  "connection_name": "shop-mongo",
  "author": "you", "description": "Orders filtered by status",
  "publish": true
}'
```

`parameters` works exactly the same as for a SQL saved query - see Use bound parameters safely for every available rule. Call it:

```shell
curl 'http://127.0.0.1:5000/q/orders_by_status?status=paid' -H 'X-API-Key: demo-key'
# [{"customer": "Carol", "total": 87.25}, {"customer": "Alice", "total": 42.5}]

curl 'http://127.0.0.1:5000/q/orders_by_status?status=refunded' -H 'X-API-Key: demo-key'
# {"error": "Invalid parameters: status must be one of: paid, pending", ...}
```

In the admin UI, this is exactly the same **API Designer → Save as New API** flow as a SQL query - pick `shop-mongo` as the connection, and the SQL editor becomes a filter/projection/sort JSON editor instead.

What's schemaless documents mean for the response shape

A collection's documents don't all have to share the same fields. QueryAPIGate builds each response's column list as the union of every key seen across the page, in first-seen order - so a query against a collection with inconsistent document shapes still produces one flat table (every existing output format - JSON, CSV, XLSX, XML, YAML - keeps working unmodified), but a document missing a field just shows that field as absent/null in its row, not an error.

Step 4: Export the whole result

A page is at most `QUERYAPIGATE_MAX_PAGE_SIZE` documents. `?stream=true` sends all of them, read from the cursor as they come, in NDJSON, CSV, TSV or Parquet:

```shell
curl 'http://127.0.0.1:5000/q/orders_by_status?status=paid&stream=true&format=ndjson' -H 'X-API-Key:
demo-key' -o paid.ndjson
queryapigate export orders_by_status --param status=paid --format parquet --out 'paid-{date}.parquet'
```

Each format handles documents that don't share fields in its own way:

- **NDJSON** writes each document whole - nested documents and arrays stay nested. Nothing is ever left out.
- **Parquet** makes every field of every document a column.

- **CSV and TSV** send their header line first, so the columns are decided before any document is written: the projection's fields, in its order - `{"customer": 1, "total": 1, "_id": 0}` above gives `customer,total` - or, without an inclusion projection, the fields of the first 1,000 documents. A field that first appears after those has no column and is left out; the run's history says how many (`fields_dropped`). Name your columns in the projection and that can't happen.

The same works ad hoc (`POST /execute_mongo?stream=true&format=csv`) and in saved exports to a bucket or a folder - there, every field becomes a column whatever the format. Exports of a MongoDB query are full exports: `incremental` isn't supported for one yet.

What's not there yet

Worth knowing up front so you don't go looking for it:

- **No writes.** `insert` / `update` / `delete` aren't reachable through QueryAPIGate at all for Mongo - only `find()`. There's no `allow_writes` equivalent to turn on.
- **No incremental exports.** A saved export of a Mongo query delivers the whole result each run.
- **No column masking.** A caller with mask rules is refused on a Mongo connection.
- **No `cache_ttl`.** Response caching (see Cache a saved query's response) isn't available for a Mongo saved query yet, only SQL ones.
- **No aggregation pipeline.** `find()` and its filter/projection/sort only - no `$group`, `$lookup`, or anything else from the aggregation framework.

Next steps

- Use bound parameters safely - the full validation-rule reference, same for Mongo and SQL saved queries alike.
- Set up your first scoped API key - a Mongo connection is scoped by a key's `connections` grant exactly like any SQL one.